

Underactuated Robotics Project

LQR Control, Trajectory Tracking, Disturbance Rejection,
Energy Shaping, MPC Concepts and Lyapunov Analysis
for a Planar Quadrotor

Daniel Nassar

December 3, 2025

Course: Underactuated Robotics

Instructor: *(to be filled)*

Institution: Lebanese American University

All simulation results, plots and animations were generated using custom Python code.

Contents

1	Introduction	1
2	System Description and Nonlinear Modeling	1
2.1	Planar Quadrotor Configuration	1
2.2	Modeling Assumptions	2
2.3	Newton–Euler Nonlinear Dynamics	2
2.4	State–Space Formulation	2
3	Equilibrium Analysis and Linearization	3
3.1	Hover Equilibrium	3
3.2	Linearization About Hover	3
4	Open–Loop Behavior	3
5	LQR Design	4
5.1	Quadratic Performance Index	4
5.2	Optimal State Feedback	4
6	LQR Stabilization Results	5
6.1	Hover Recovery from a Perturbed State	5
6.2	Control Input Behavior	5
6.3	Stabilization Animation	6
7	Reference Trajectory Tracking	6
7.1	LQR-Based Tracking Law	6
7.2	Smooth Horizontal Repositioning (Scenario 0)	7
7.3	Scenario-Based Trajectory Tracking	7
8	Disturbance Rejection with Unknown Wind	11
9	Lyapunov Stability and Region of Attraction	12
9.1	Lyapunov Function from the Riccati Matrix	12
9.2	Approximate Region of Attraction in (x, θ)	12
10	Energy Shaping and Passivity Concepts	14
11	Model Predictive Control (MPC) Perspective	14
12	Discussion and Future Work	15
13	Conclusion	15

List of Figures

1	Open-loop nonlinear simulation. A small initial perturbation causes the planar quadrotor to drift away from the hover configuration: altitude deviates from z^* and the horizontal position keeps moving. This motivates the need for feedback control.	4
2	Closed-loop state trajectories under LQR hover stabilization. The horizontal position x is driven to zero, the altitude z converges to the desired value, and the pitch angle θ returns to zero with a small transient.	5
3	LQR control inputs during hover recovery. The total thrust F_T quickly settles near mg , and the torque τ decays to zero, indicating moderate actuator effort.	6
4	Snapshot from the LQR stabilization animation. The quadrotor starts off-center with a small pitch and then smoothly returns to the hover configuration.	6
5	LQR tracking of a smooth horizontal repositioning maneuver (Scenario 0). The actual position $x(t)$ tracks the S-curve reference, while altitude and pitch stay close to their desired values.	7
6	Tracking errors for Scenario 0. Errors in x , z and θ converge to zero with modest overshoot in x	8
7	States versus reference for sinusoidal tracking (Scenario 1). The quadrotor follows $x_{\text{ref}}(t) = A \sin(\omega t)$ while maintaining constant altitude and zero pitch on average.	9
8	Tracking errors for the sinusoidal reference. The errors remain bounded and relatively small, demonstrating good performance as long as the amplitude stays inside the linearization region.	10
9	Closed-loop states in the presence of an unknown wind disturbance. When the wind is applied, the quadrotor is pushed horizontally, but the LQR controller reorients the body and brings the state back toward hover.	11
10	Control inputs under wind. Both thrust and torque exhibit a transient response as the controller counteracts the disturbance, then return close to their nominal hover values.	11
11	Lyapunov function $V(t) = \tilde{x}^T P \tilde{x}$ along a nonlinear LQR trajectory. The function is nonnegative and decreases monotonically to zero (up to numerical noise), providing strong evidence of nonlinear stability in a neighborhood of the hover equilibrium.	12
12	Lyapunov level sets in the (x, θ) plane. Colored ellipses correspond to different values of V , and a sample closed-loop trajectory is superimposed. The trajectory spirals inward while remaining inside the outer level sets, then enters an inner “fast convergence” region, offering an intuitive picture of the ROA slice.	13

1 Introduction

Quadrotor vehicles are a standard benchmark in underactuated robotics: they are mechanically simple but dynamically rich, and they capture many of the challenges of controlling real flying robots. In this project we study a *planar* quadrotor, sometimes called a 2D VTOL (Vertical Take-Off and Landing) system. The motion is restricted to the vertical x - z plane, so the configuration is described by three variables: horizontal position x , vertical position z , and pitch angle θ . The system is *underactuated* because it has three degrees of freedom but only two independent control inputs (total thrust and body torque).

The project has both theoretical and practical goals:

- derive the nonlinear Newton–Euler model of the planar quadrotor;
- compute the hover equilibrium and obtain a linearization;
- design a Linear Quadratic Regulator (LQR) for local stabilization;
- extend the same LQR gain to perform reference trajectory tracking;
- study robustness against an unknown wind disturbance;
- perform Lyapunov analysis using the LQR Riccati matrix and visualize an approximate region of attraction (ROA);
- discuss, at a theoretical level, how energy shaping and Model Predictive Control (MPC) could further improve performance.

All numerical simulations and plots are implemented in Python using a modular code structure: `dynamics.py` (nonlinear model), `simulation.py` (ODE integration), `controllers.py` (LQR design), and `visualization.py` (plots and animations). The focus is not on using a specific tool such as MATLAB or Drake, but on understanding the control concepts deeply and implementing them from scratch.

2 System Description and Nonlinear Modeling

2.1 Planar Quadrotor Configuration

The planar quadrotor consists of a rigid bar of total length 2ℓ with two rotors at its ends, as shown conceptually in fig. 1. The rigid body has mass m and rotational inertia I about its center of mass (COM). The configuration variables are:

- x : horizontal position of the COM in the inertial frame;
- z : vertical position of the COM;
- θ : pitch angle, measured from the horizontal axis, positive when the right rotor goes up.

The generalized coordinate vector is

$$q = [x \quad z \quad \theta]^\top.$$

Each rotor produces an upward thrust F_1 and F_2 in the body frame. In the planar model these forces are co-linear and always perpendicular to the bar. It is convenient to define the total thrust and the net torque:

$$F_T = F_1 + F_2, \tag{1}$$

$$\tau = \ell(F_2 - F_1). \tag{2}$$

2.2 Modeling Assumptions

To simplify the dynamics while preserving the key underactuated features, the following assumptions are made:

- motion is restricted to the x - z plane, i.e., no lateral motion;
- the body is rigid, symmetric, and the COM is centered;
- thrust forces act along the body z -axis and pass through the rotors without offset;
- aerodynamic drag, rotor dynamics and saturations are neglected;
- mass m and inertia I are constant;
- gravity acts in the $-z$ direction with magnitude g .

Although these assumptions idealize the hardware, they are typical in underactuated robotics courses and allow a clean control design pipeline.

2.3 Newton–Euler Nonlinear Dynamics

Applying Newton’s second law in the inertial frame with the chosen sign conventions, we obtain the translational dynamics

$$m\ddot{x} = -F_T \sin \theta, \quad (3)$$

$$m\ddot{z} = F_T \cos \theta - mg, \quad (4)$$

and the rotational dynamics

$$I\ddot{\theta} = \tau. \quad (5)$$

The equations show the underactuation clearly:

- the total thrust F_T is used primarily to control z ,
- the torque τ controls θ ,
- horizontal motion x is affected indirectly through the tilt angle θ .

To move in the x direction, the quadrotor must first tilt, so that the vertical thrust vector acquires a horizontal component.

2.4 State–Space Formulation

We define the state vector

$$x_s = [x \quad \dot{x} \quad z \quad \dot{z} \quad \theta \quad \dot{\theta}]^T$$

and the input vector

$$u = [F_T \quad \tau]^T.$$

Using eqs. (3) to (5), the nonlinear state–space model is

$$\dot{x}_s = f(x_s, u) = \begin{bmatrix} \dot{x} \\ -\frac{F_T}{m} \sin \theta \\ \dot{z} \\ \frac{F_T}{m} \cos \theta - g \\ \dot{\theta} \\ \frac{\tau}{I} \end{bmatrix}. \quad (6)$$

This model is the only one used in simulation; linearization is employed solely for controller design and analysis.

3 Equilibrium Analysis and Linearization

3.1 Hover Equilibrium

A state–input pair (x_s^*, u^*) is an equilibrium if $f(x_s^*, u^*) = 0$. Hover means the quadrotor is at constant position with zero velocity and zero pitch angle:

$$x_s^* = [x^* \ 0 \ z^* \ 0 \ 0 \ 0]^\top,$$

where x^* and z^* are arbitrary constants. Substituting this into the dynamics gives

$$\theta^* = 0, \tag{7}$$

$$F_T^* = mg, \tag{8}$$

$$\tau^* = 0. \tag{9}$$

Thus at hover, the total thrust balances gravity and the torque is zero.

3.2 Linearization About Hover

Let $\tilde{x} = x_s - x_s^*$ and $\tilde{u} = u - u^*$ denote deviations from equilibrium. The linearized model is

$$\dot{\tilde{x}} = A\tilde{x} + B\tilde{u}, \tag{10}$$

where

$$A = \left. \frac{\partial f}{\partial x_s} \right|_{(x_s^*, u^*)}, \quad B = \left. \frac{\partial f}{\partial u} \right|_{(x_s^*, u^*)}. \tag{11}$$

Evaluating the Jacobians at $\theta = 0$ and $F_T = mg$ yields

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & -g & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 0 & 0 \\ 1/m & 0 \\ 0 & 0 \\ 0 & 1/I \end{bmatrix}. \tag{12}$$

The linear model captures local behavior near hover. Because the LQR gain is computed from eq. (12), its performance is guaranteed only for moderate deviations from the equilibrium; large tilts or aggressive reference commands can violate the linear approximation and degrade tracking performance.

4 Open–Loop Behavior

Before designing any controller it is instructive to observe the open–loop dynamics. The quadrotor is simulated with constant hover inputs $u = u^*$ but with a small perturbation in initial position and angle. As expected, the system does not naturally return to hover.

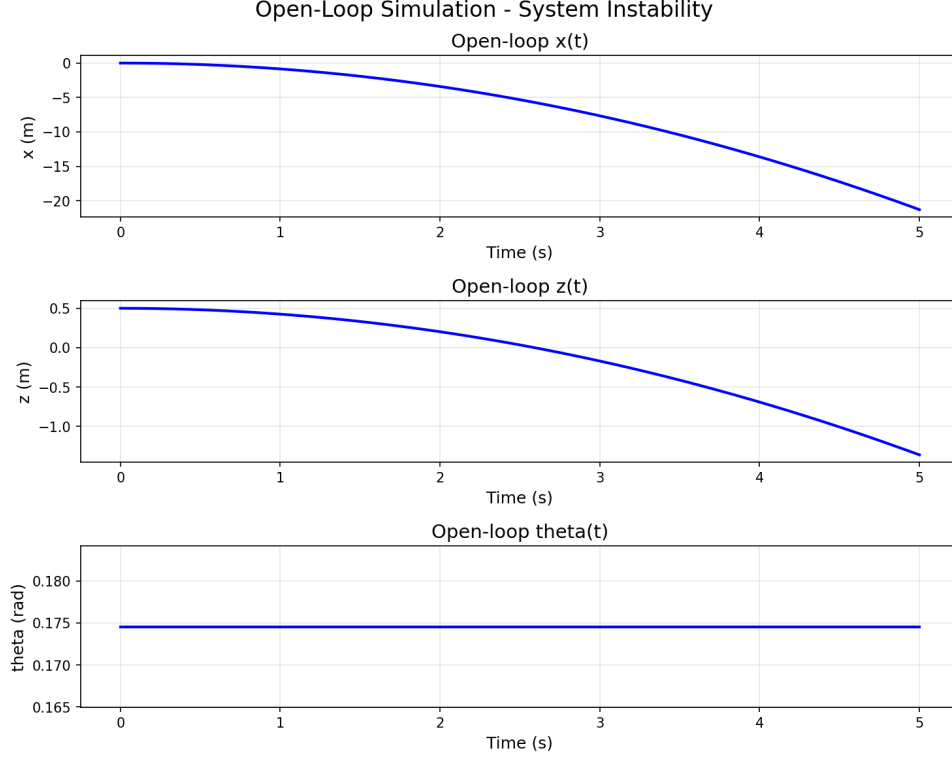


Figure 1: Open-loop nonlinear simulation. A small initial perturbation causes the planar quadrotor to drift away from the hover configuration: altitude deviates from z^* and the horizontal position keeps moving. This motivates the need for feedback control.

5 LQR Design

5.1 Quadratic Performance Index

For the linear system, the LQR controller minimizes the infinite-horizon cost

$$J = \int_0^\infty \left(\tilde{x}(t)^\top Q \tilde{x}(t) + \tilde{u}(t)^\top R \tilde{u}(t) \right) dt, \quad (13)$$

with symmetric weighting matrices $Q \succeq 0$ and $R \succ 0$. The chosen weights are

$$Q = \text{diag}(1, 0.1, 10, 5, 100, 10), \quad (14)$$

$$R = \text{diag}(1, 1). \quad (15)$$

Intuitively:

- altitude z and pitch angle θ are heavily penalized to keep the platform upright and at the correct height;
- horizontal position x is penalized but less aggressively;
- actuator effort is kept moderate via the entries in R .

5.2 Optimal State Feedback

The optimal linear feedback is

$$\tilde{u} = -K\tilde{x}, \quad (16)$$

where K is obtained from the solution of the continuous Riccati equation

$$A^T P + P A - P B R^{-1} B^T P + Q = 0. \quad (17)$$

Using the `python-control` library, the resulting gain is

$$K = \begin{bmatrix} -8.79 \cdot 10^{-16} & 6.40 \cdot 10^{-16} & 3.16 & 3.37 & -1.22 \cdot 10^{-15} & -6.38 \cdot 10^{-18} \\ -1.00 & -1.75 & -1.01 \cdot 10^{-15} & -3.19 \cdot 10^{-16} & 14.54 & 3.25 \end{bmatrix}. \quad (18)$$

In original coordinates, the control law becomes

$$u(t) = u^* - K(x_s(t) - x_s^*). \quad (19)$$

6 LQR Stabilization Results

6.1 Hover Recovery from a Perturbed State

To test the stabilizing effect, the closed loop is simulated with an initial state

$$x_s(0) = [0.2 \ 0 \ 0.8 \ 0 \ 0.15 \ 0]^T,$$

representing a horizontal offset, a higher altitude than desired, and a small pitch angle. The nonlinear dynamics are integrated with the LQR control law applied at each time step.

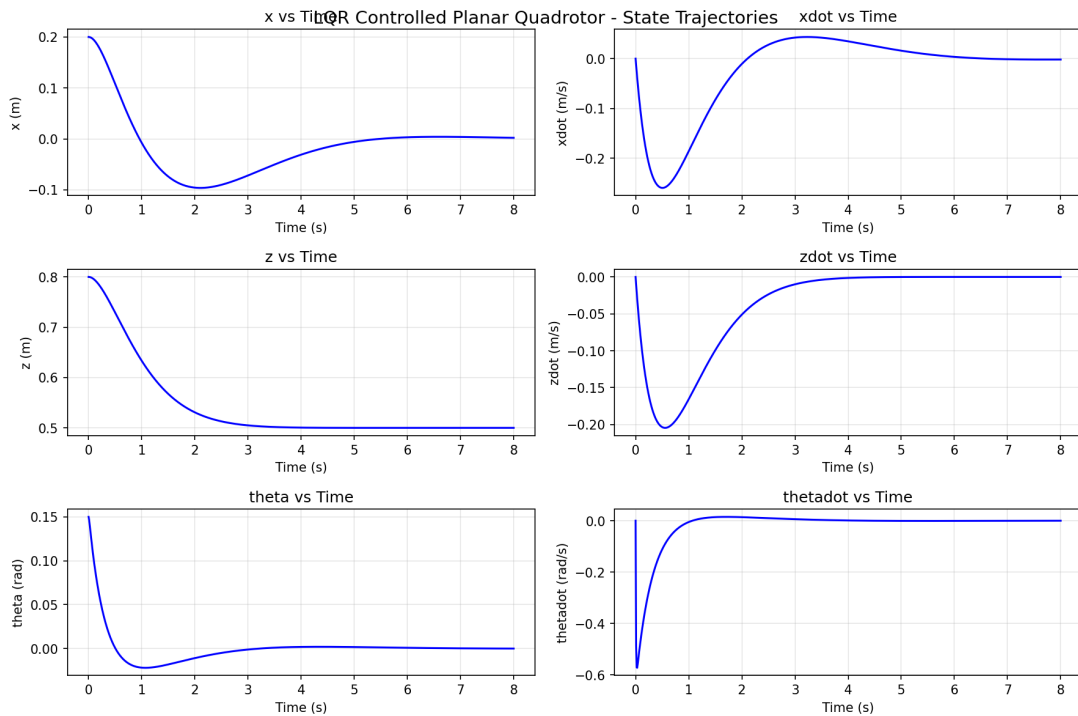


Figure 2: Closed-loop state trajectories under LQR hover stabilization. The horizontal position x is driven to zero, the altitude z converges to the desired value, and the pitch angle θ returns to zero with a small transient.

6.2 Control Input Behavior

Figure 3 shows the total thrust and torque used during hover recovery. The thrust quickly stabilizes near mg while the torque returns to zero. The peaks remain within reasonable bounds, illustrating that the chosen Q and R achieve a reasonable trade-off between speed and effort.

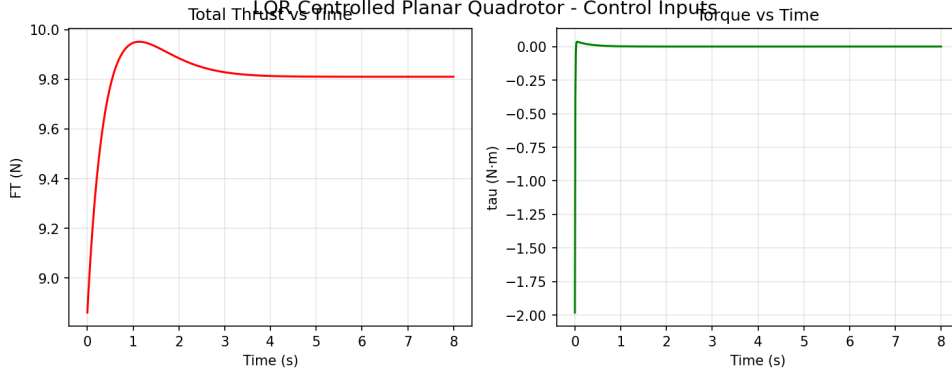


Figure 3: LQR control inputs during hover recovery. The total thrust F_T quickly settles near mg , and the torque τ decays to zero, indicating moderate actuator effort.

6.3 Stabilization Animation

To make the behavior intuitive for presentation, an animation visualizes the body segment, rotors, and center of mass. A dashed gray curve shows the path of the COM as it returns to hover.

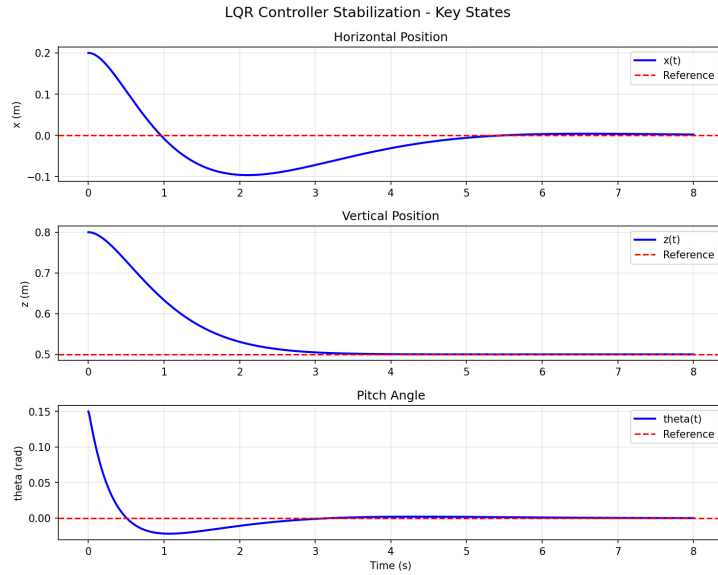


Figure 4: Snapshot from the LQR stabilization animation. The quadrotor starts off-center with a small pitch and then smoothly returns to the hover configuration.

7 Reference Trajectory Tracking

7.1 LQR-Based Tracking Law

The same LQR gain can be reused to track a time-varying reference $x_{\text{ref}}(t)$ by treating the error $x_s - x_{\text{ref}}(t)$ as the new deviation:

$$u(t) = u^* - K(x_s(t) - x_{\text{ref}}(t)). \quad (20)$$

The reference is specified for the linearized model, but the true dynamics remain nonlinear in simulation. This is similar to gain-scheduled LQR tracking and works well as long as the reference stays in the linear region.

7.2 Smooth Horizontal Repositioning (Scenario 0)

In the first tracking experiment the quadrotor must move smoothly from $x = 0$ to $x = 0.5$ m while keeping $z = 0.5$ m and $\theta = 0$. A cosine S-curve is used to generate a reference with continuous position and velocity.

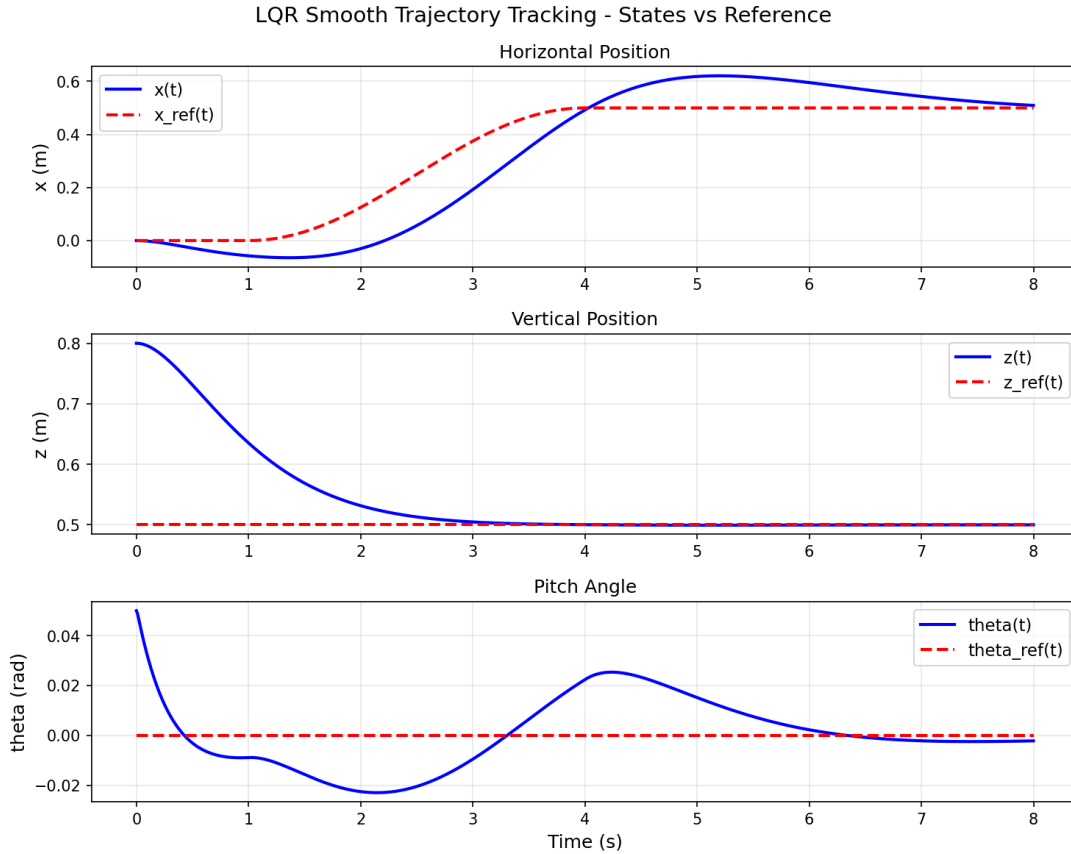


Figure 5: LQR tracking of a smooth horizontal repositioning maneuver (Scenario 0). The actual position $x(t)$ tracks the S-curve reference, while altitude and pitch stay close to their desired values.

The corresponding animation shows the reference point (red star) sliding horizontally while the quadrotor body follows with a small, physically plausible pitch.

7.3 Scenario-Based Trajectory Tracking

To further challenge the controller, several reference generators were implemented:

- **Scenario 1:** sinusoidal motion in x ;
- **Scenario 2:** ramp movement with saturation;
- **Scenario 3:** logistic S-curve;
- **Scenario 4:** square wave between two positions.

Importantly, *the same* LQR gain K is used in all scenarios; only the reference trajectory changes.

Animations with the reference path drawn as a dashed red line are especially useful during presentations: they make the abstract tracking problem look like an intuitive pursuit of a moving target.

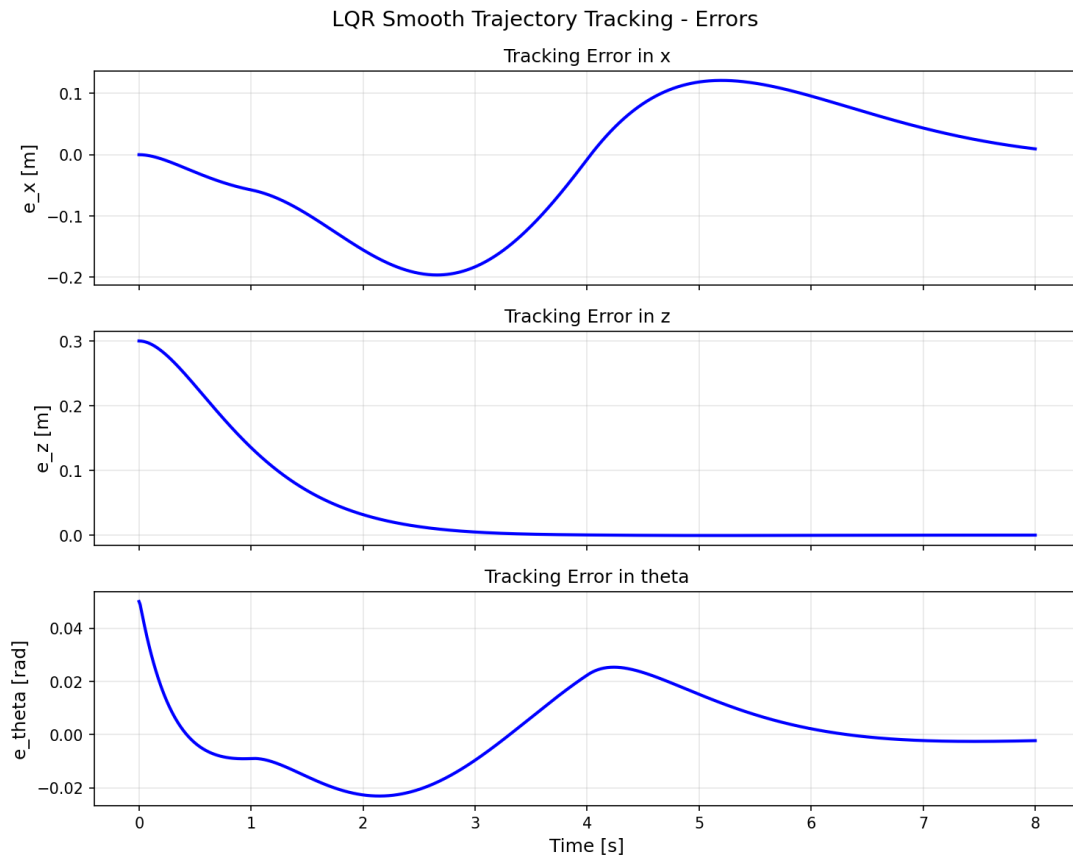


Figure 6: Tracking errors for Scenario 0. Errors in x , z and θ converge to zero with modest overshoot in x .

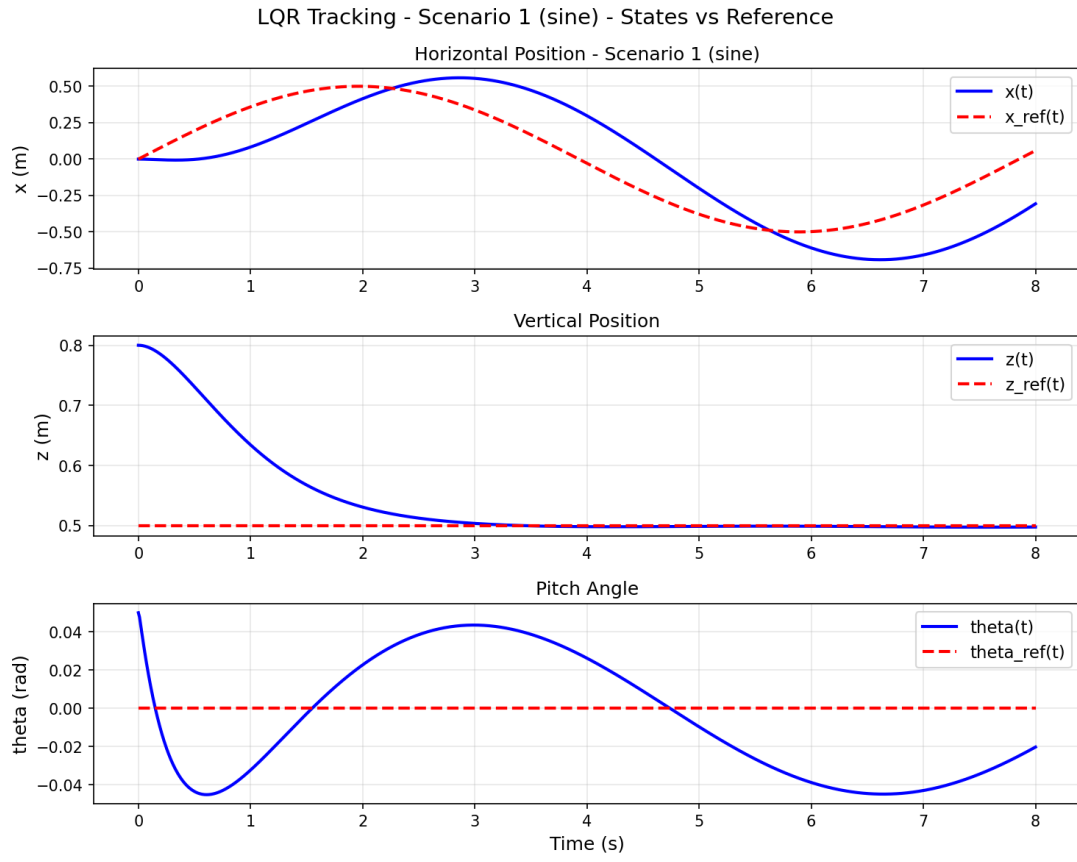


Figure 7: States versus reference for sinusoidal tracking (Scenario 1). The quadrotor follows $x_{ref}(t) = A \sin(\omega t)$ while maintaining constant altitude and zero pitch on average.

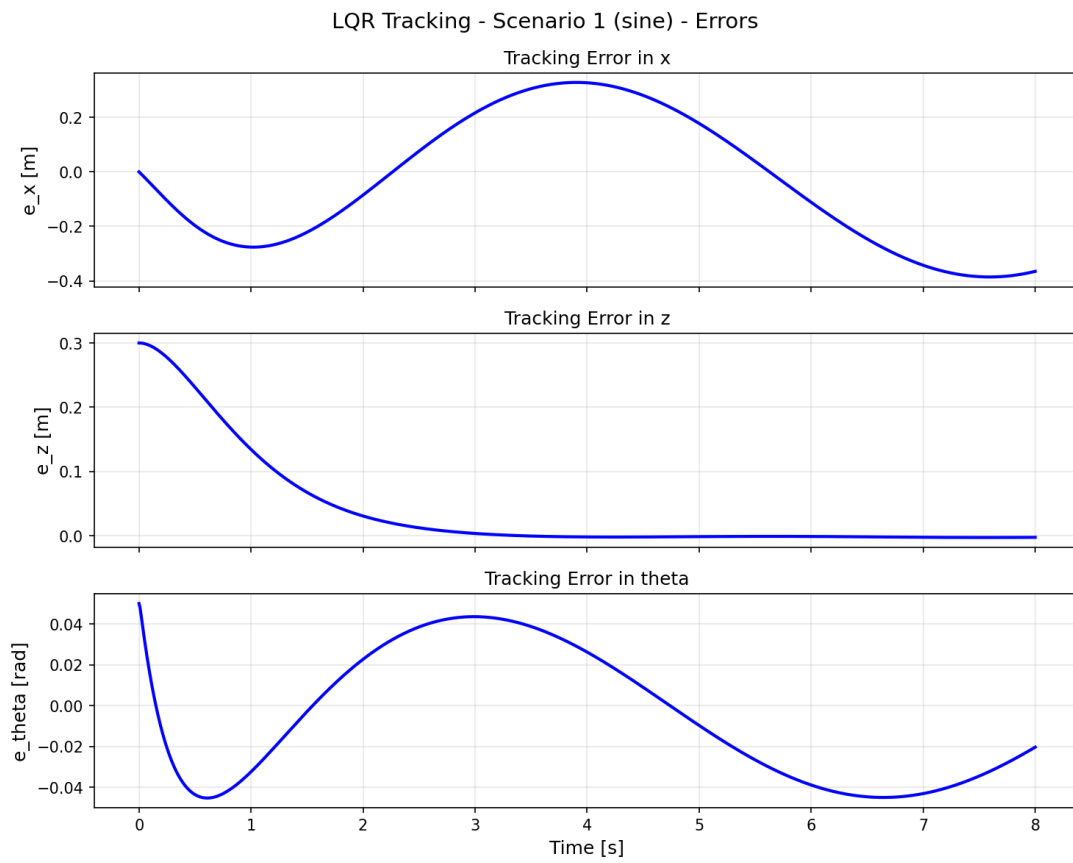


Figure 8: Tracking errors for the sinusoidal reference. The errors remain bounded and relatively small, demonstrating good performance as long as the amplitude stays inside the linearization region.

8 Disturbance Rejection with Unknown Wind

Real quadrotors are subject to disturbances such as wind gusts. To test robustness, a horizontal “wind” force F_{wind} is injected into the x -dynamics at the nonlinear level:

$$m\ddot{x} = -F_T \sin \theta + F_{\text{wind}}(t), \quad (21)$$

where $F_{\text{wind}}(t)$ is a step or pulse applied after $t = 2$ s. Crucially, this disturbance is *not* modeled in the LQR design; the controller simply observes that the state has drifted from the desired hover and reacts accordingly.

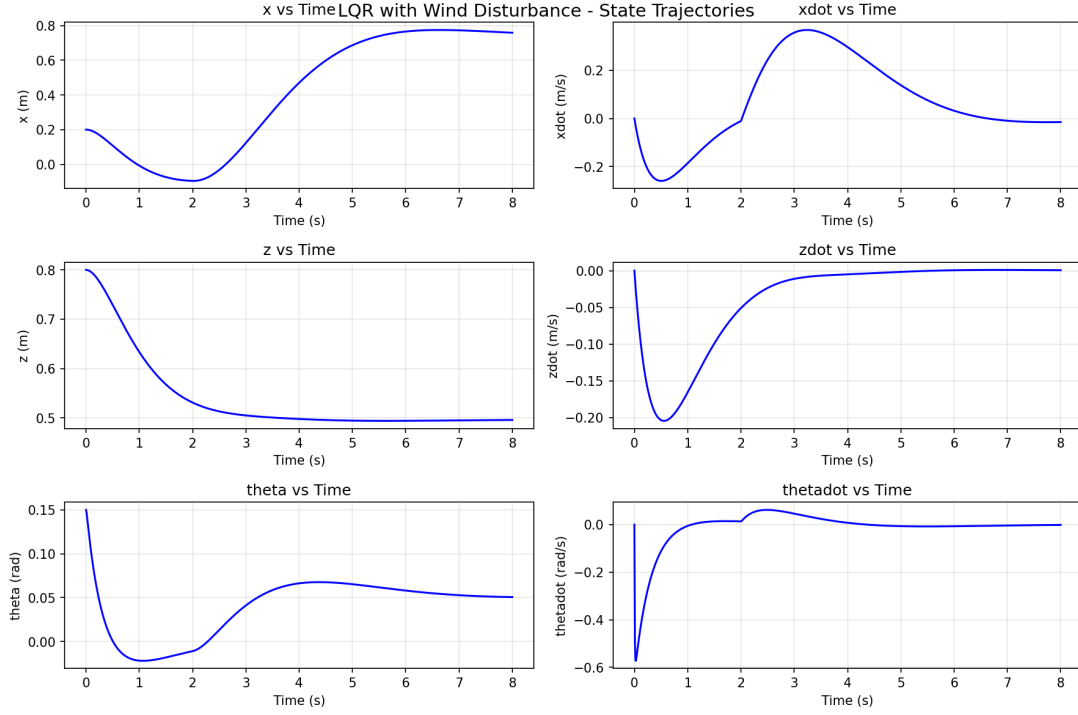


Figure 9: Closed-loop states in the presence of an unknown wind disturbance. When the wind is applied, the quadrotor is pushed horizontally, but the LQR controller reorients the body and brings the state back toward hover.

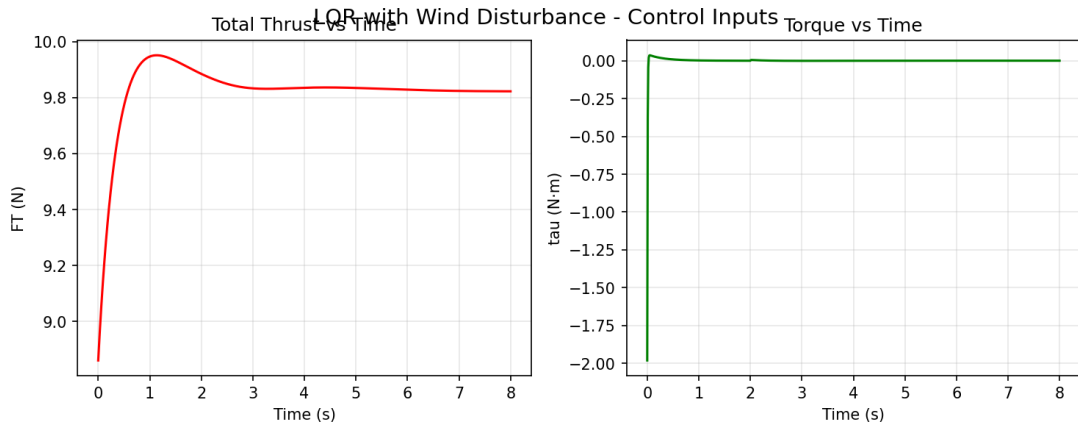


Figure 10: Control inputs under wind. Both thrust and torque exhibit a transient response as the controller counteracts the disturbance, then return close to their nominal hover values.

The associated animation clearly shows the disturbance: the quadrotor is pushed sideways,

tilts against the wind, and then slides back to the original hover position. This experiment confirms that LQR, although designed from a nominal model, offers a useful degree of disturbance rejection for moderate perturbations.

9 Lyapunov Stability and Region of Attraction

9.1 Lyapunov Function from the Riccati Matrix

For the linear closed-loop system

$$\dot{\tilde{x}} = (A - BK)\tilde{x},$$

the Riccati matrix P used in the LQR design is positive definite, and the quadratic form

$$V(\tilde{x}) = \tilde{x}^\top P \tilde{x} \quad (22)$$

is a natural Lyapunov candidate. Along trajectories,

$$\dot{V}(\tilde{x}) = \tilde{x}^\top [(A - BK)^\top P + P(A - BK)] \tilde{x} \quad (23)$$

$$= -\tilde{x}^\top (Q + K^\top R K) \tilde{x} \leq 0. \quad (24)$$

Thus the origin is asymptotically stable for the linear model. To test whether this behavior persists for the true nonlinear system, $V(t)$ is evaluated along a nonlinear closed-loop trajectory.

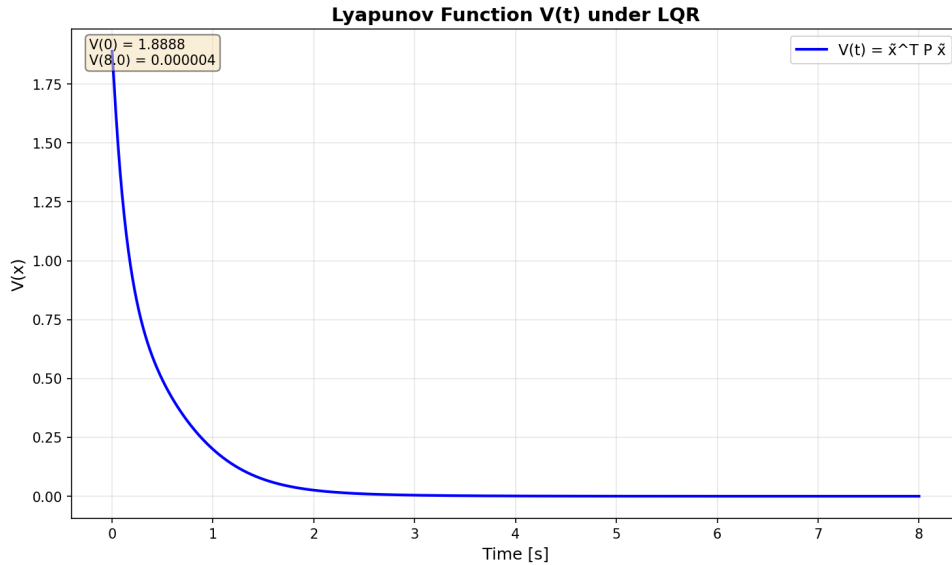


Figure 11: Lyapunov function $V(t) = \tilde{x}^\top P \tilde{x}$ along a nonlinear LQR trajectory. The function is nonnegative and decreases monotonically to zero (up to numerical noise), providing strong evidence of nonlinear stability in a neighborhood of the hover equilibrium.

9.2 Approximate Region of Attraction in (x, θ)

A full region of attraction (ROA) in the six-dimensional state space is hard to visualize. Instead, level sets $V(\tilde{x}) = c$ are plotted in the (x, θ) plane while the other states are fixed at their equilibrium values. Each contour is an ellipse corresponding to a constant “energy” level.

The outermost ellipse where simulations still converge can be interpreted as a conservative approximation of the ROA in this two-dimensional slice. In principle, one could search for the largest sublevel set $V(\tilde{x}) \leq c$ such that $\dot{V}(\tilde{x}) < 0$ for all states on the boundary; this would yield a mathematically rigorous ROA estimate.

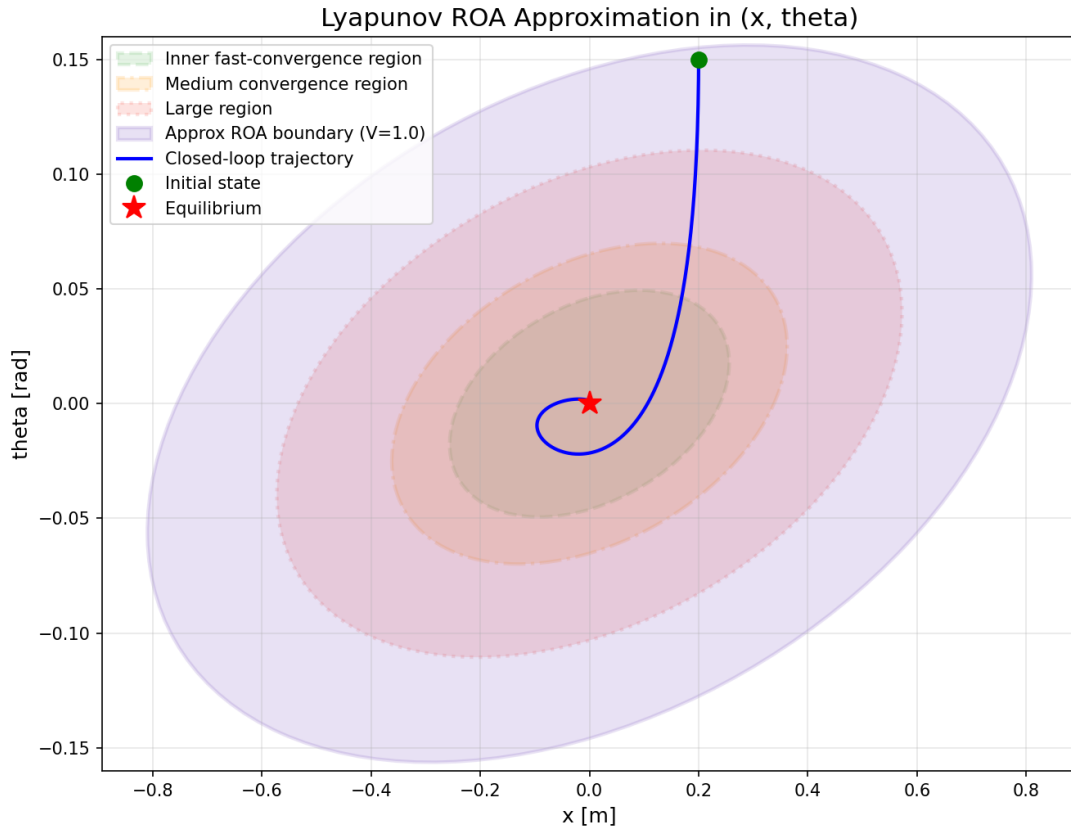


Figure 12: Lyapunov level sets in the (x, θ) plane. Colored ellipses correspond to different values of V , and a sample closed-loop trajectory is superimposed. The trajectory spirals inward while remaining inside the outer level sets, then enters an inner “fast convergence” region, offering an intuitive picture of the ROA slice.

10 Energy Shaping and Passivity Concepts

Although the implemented controller is purely LQR, the planar quadrotor also lends itself to *energy shaping* techniques, which are common in underactuated robotics (e.g., for the Acrobot or cart–pole systems). The basic idea is to view the system as a mechanical system with kinetic energy T and potential energy V_g due to gravity, then design control inputs that modify the effective potential or damping.

For the planar quadrotor, the physical energy is approximately

$$E = T + V_g = \frac{1}{2}m(\dot{x}^2 + \dot{z}^2) + \frac{1}{2}I\dot{\theta}^2 + mgz. \quad (25)$$

Near hover, an energy–based controller could:

- inject or remove energy to regulate altitude z ,
- shape the potential so that the desired configuration becomes a minimum of the total energy,
- add damping to guarantee convergence.

In underactuated systems such as the cart–pole, energy shaping is often used to swing the pendulum up by driving the energy to a desired value, and then a local LQR controller stabilizes around the upright position. A similar strategy could be envisioned for aggressive maneuvers of the quadrotor (e.g., moving between two hover points by exploiting its mechanical energy). Due to time constraints, only the LQR part was implemented in code, but the project report highlights how energy shaping fits conceptually into the control toolbox for underactuated robots.

11 Model Predictive Control (MPC) Perspective

Model Predictive Control (MPC) is a powerful alternative to LQR, especially when input and state constraints must be respected. At each sampling time, MPC solves a finite–horizon optimal control problem for a discrete–time model of the system and applies the first input of the optimal sequence. The process is repeated at the next time step.

For the planar quadrotor, an MPC formulation could:

- discretize the linear model $\tilde{x}_{k+1} = A_d\tilde{x}_k + B_d\tilde{u}_k$;
- impose bounds on thrust and torque (e.g., $F_T^{\min} \leq F_T \leq F_T^{\max}$);
- limit the maximum tilt angle $|\theta| \leq \theta_{\max}$ for safety;
- penalize deviations from a reference trajectory over a prediction horizon N using the same quadratic stage cost as LQR.

Compared to the implemented LQR, MPC would:

- explicitly handle constraints (useful for large disturbances or aggressive tracking commands);
- naturally incorporate preview information about future references;
- require solving an optimization problem online at each step, which is more computationally demanding than a simple matrix multiplication.

In this project, MPC is discussed at the conceptual level rather than implemented, because the existing LQR controller already provides strong results and the focus of the course is on understanding underactuated dynamics and Lyapunov analysis. However, the same Python framework could be extended in the future using libraries such as `cvxpy` or `osqp` to implement a real MPC controller and quantitatively compare it with LQR.

12 Discussion and Future Work

The project developed a complete pipeline for controlling an underactuated planar quadrotor:

- from the nonlinear Newton–Euler equations,
- through linearization and LQR design,
- to practical simulations with tracking and disturbances,
- and finally Lyapunov–based stability analysis.

Key observations include:

- The LQR controller robustly stabilizes hover even when the initial state is noticeably perturbed.
- The same gain can be reused to track a variety of horizontal reference trajectories, as long as they lie within the linear region around hover.
- When a wind disturbance is injected at the dynamics level, the LQR controller is still able to re-stabilize the system, effectively rejecting the perturbation.
- The Riccati matrix provides a natural Lyapunov function for both proving local stability of the linear model and approximating a region of attraction in the nonlinear system.

Natural extensions of the project include:

- implementing energy–shaping controllers for more aggressive maneuvers (e.g., large swings or flips) and combining them with LQR for local stabilization;
- designing and testing an MPC controller to compare constraint handling and performance against LQR;
- extending the model to full 3D quadrotor dynamics with additional underactuated degrees of freedom.

13 Conclusion

This report has presented a detailed study of a planar quadrotor as an underactuated robotic system. Starting from first principles, we derived the nonlinear equations of motion and linearized them around a hover equilibrium. An LQR controller was designed and implemented in Python to stabilize the system, track several reference trajectories, and reject wind disturbances.

A Lyapunov function constructed from the LQR Riccati matrix was used to evaluate stability along nonlinear trajectories and to visualize an approximate region of attraction. Although only LQR was implemented in practice, the report also discussed how energy shaping and MPC could further enhance performance and handle constraints.

Overall, the project demonstrates that even with a simplified 2D model, the planar quadrotor provides a rich and educational example of underactuated control, combining modeling, optimal control, Lyapunov theory, and numerical simulation in a coherent framework.